Stolpe,
GraalLSP

GraalLSP: A Zero-overhead Polyglot Language Server Implementation for the GraalVM

GraalLSP: Eine Overhead-freie Language Server Implementierung für die GraalVM

by

Daniel Stolpe

A thesis submitted to the
Hasso Plattner Institute
at the University of Potsdam, Germany
in partial fulfillment of the requirements for the degree of

Master of Science in IT-Systems Engineering

Supervisors

Prof. Dr. Robert Hirschfeld
Dr. John Doe

Software Architecture Group
Hasso Plattner Institute
University of Potsdam, Germany

October 10, 2018

Abstract

The english abstract.

Zusammenfassung

Die Zusammenfassung auf deutsch.

Acknowledgments

I owe everything to my cat.

Contents

1. Introduction **1**
1.1. Challanges of developing your own language . . . 1
1.2. Language implementations - performance vs. tooling support . . . 1
1.3. Contribution of this Thesis: GraalLSP . . . 1
1.4. Structur of this Thesis . . . 1

2. Context **3**
2.1. Graal and Truffle . . . 3
2.1.1. GraalVM . . . 3
2.1.2. Instrumentation . . . 3
2.2. Language Server Protocol . . . 4

3. Approach **5**
3.1. How to get the relevant information out of Graal/Truffle . . . 5
3.1.1. Parsing code . . . 5
3.1.2. Mapping editor cursor position to Truffle source-section and node . . . 5
3.1.3. Instrumentation . . . 5
3.1.4. Challanges due to Truffles focus on performance . . . 5
3.2. Type harvesting evaluation strategies . . . 5
3.2.1. Literals . . . 5
3.2.2. Coverage data . . . 5
3.2.3. Run-to-section . . . 5
3.2.4. Globals . . . 5
3.3. LSP features overview table . . . 5

4. Implementation **7**
4.1. The lsp4j library . . . 7
4.2. Architecture . . . 7
4.2.1. Truffle-API + Graal-SDK . . . 7
4.2.2. Launcher, Instrument, Server . . . 7
4.2.3. Thread with entered Context . . . 7
4.3. What a TruffleLanguage needs to implement . . . 7
4.3.1. Expression- and Statement-Tag . . . 8
4.3.2. findSourceLocation() . . . 8
4.3.3. findTopScopes() . . . 8
4.3.4. findLocalScopes() . . . 8
4.3.5. findMetaObject() . . . 8
4.3.6. getCompletionTriggerCharacters() . . . 8

4.3.7. boxPrimitive() . . . 8
4.3.8. parseInline() . . . 8
4.3.9. GetSignature custom message . . . 8
4.4. LSP features implementation - TextDocumentService . . . 8
4.5. LSP features implementation - WorkspaceService . . . 8
4.6. LSP client config . . . 8
4.6.1. VSCode extension exmaple + Limitations . . . 8

5. Experience report **9**
5.1. Between IDE and Language Workbench . . . 9
5.2. Completion response time . . . 9
5.3. Limitations . . . 9
5.3.1. VSCode: Need to define supported languages in extension . . . 9
5.3.2. VSCode: Syntax highlighting defined at client-side . . . 9
5.3.3. Server: Only TCP (connecting to already running server), no std-in/-out impl . . . 9
5.3.4. Truffle: Only first syntax error is reported. Lang parsers fail on first error . . . 9

6. Related Work **11**
6.1. Graal . . . 11
6.2. Squimera . . . 11
6.3. Language Workbenches . . . 11
6.4. Run-time type collection . . . 11

7. Conclusion and Future Work **13**

A. First Unimportant stuff. **17**

List of Figures

2.1. example for how a tool and a language server communicate during a routine editing session 4

List of Tables

List of Listings

List of Abbreviations

API application programming interface
LSP Language Server Protocol

1. Introduction

1.1. Challanges of developing your own language

User groups: language developer and language user

For a simple example, consider if we want to rename the properties on the target class in my simple example. With a first class modern IDE, automatic refactorings for renaming are habitual. But such a rename won't propagate into the DSL. There is what I'll call a symbolic barrier between the world of C# and the file mapping DSL. We can translate our mapping into C#, but the barrier limits our ability to manipulate the overall program. This lack of integration hits us in lots of ways with tooling. Firstly - how do we edit our DSL? A text editor will do the job - but modern IDEs increasingly make text editors look primitive. I should get a pop-up list and completion on the field names, red squiggles if the character ranges overlap. But to do this I need an editor that understands the semantics of my DSL. Maybe I can live without a semantic editor. But then think about debugging. My debugger can step into the C# translations, but can't get into the true source itself. What I really would like is a full-blown IDE for my DSL. In the days of text editors and simple debuggers this wasn't a big issue - but we now live in a post-IntelliJ world.

1.2. Language implementations - performance vs. tooling support

Graal/Truffle: Performance first [2]. Tooling is enabled through instrumentation [9].

1.3. Contribution of this Thesis: GraalLSP

This thesis proposes the idea to build a common language server for programming languages implemented with the Truffle framework. For the language developer: Reduce the amount of work (can focus on language development, not tool development) For the language user: Broader choice in tooling (any LSP client) + more consistent tooling experience for all Graal/Truffle-implemented languages

1.4. Structur of this Thesis

2. Context

2.1. Graal and Truffle

2.1.1. GraalVM

GraalVM[7] is a high-performance polyglot VM.

GraalVM is a universal virtual machine for running applications written in JavaScript, Python 3, Ruby, R, JVM-based languages like Java, Scala, Kotlin, and LLVM-based languages such as C and C++.

GraalVM removes the isolation between programming languages and enables interoperability in a shared runtime. It can run either standalone or in the context of OpenJDK, Node.js, Oracle Database, or MySQL.

You can use Truffle to implement your own new programming language, to create a high-performance implementation of an existing programming language, or to implement a domain specific language. We talk a lot about the details of Truffle and Graal in our project, but we often forget to mention that Truffle is the easy way to implement a language. And you get features like the debugger automatically. Anyone who's done just an undergraduate course in programming language implementation should have the basic skills needed. Oracle Labs implemented a basic version of Ruby faster than any previous efforts with just one intern over a few months.

2.1.2. Instrumentation

Implement your own Tools

GraalVM provides a framework for creating language-agnostic tools like debuggers, profilers, or other instrumentations. In general, GraalVM provides a standardized way to express and run program code enabling cross-language research and the development of tools that are developed once and then can be applied to any language.

Create Language-agnostic Tools

GraalVM provides a framework for creating language-agnostic tools like debuggers, profilers, or other instrumentations. GraalVM provides a standardized way to express and run program code enabling cross-language research and the development of tools that are developed once and then can be applied to any language.

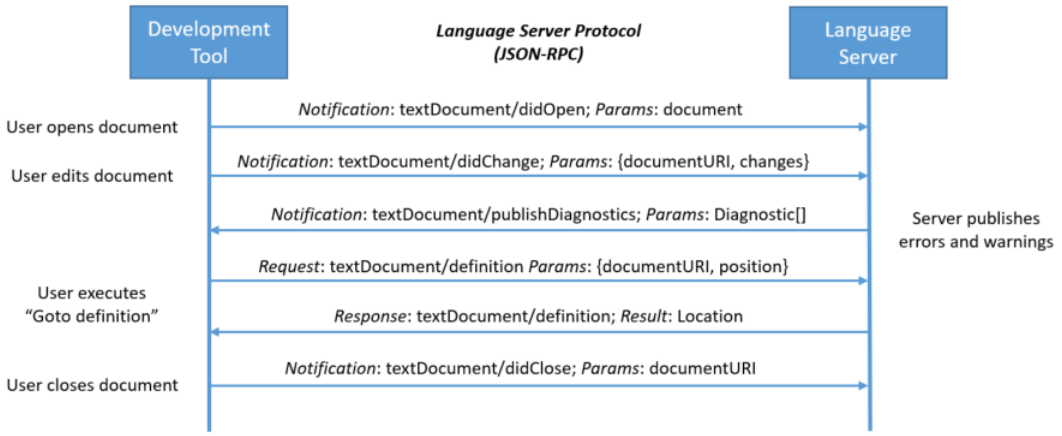


Figure 2.1.: example for how a tool and a language server communicate during a routine editing session

2.2. Language Server Protocol

In 2016 Microsoft started to standardize the Language Server Protocol (LSP)[4]. With this protocol, Microsoft tackles the problem of language and tool developers, which need to repeatedly implement language support (e.g. code completion, jump-to-definition and syntax error reporting) for every new editor/IDE. The LSP defines a *language server* and a *language client*. The server needs to be implemented only once for every language, providing the language-specific smarts. A development tool implementing a *language client* can make use of any *language server* and therefore support multiple languages with minial effort.

Today, language servers for more than 50 languages exist[1], including almost all popular programming and scripting languages[2]. Not all implementations support the full feature set defined in the LSP. Typically, these language server implementations support exactly one language, but the LSP allows multiple languages per server.

[1] https://microsoft.github.io/language-server-protocol/implementors/servers/ Sep. 2018
[2] Regarding the popularity on GitHub in 2017 https://octoverse.github.com/

3. Approach

3.1. How to get the relevant information out of Graal/Truffle

3.1.1. Parsing code

3.1.2. Mapping editor cursor position to Truffle source-section and node

3.1.3. Instrumentation

3.1.4. Challanges due to Truffles focus on performance

AST nodes lack relevant information, e.g. for refactoring. The language parsers are designed as fail-on-first-error. Only first syntax error is reported.

3.2. Type harvesting evaluation strategies

Is this already implementation?

3.2.1. Literals

3.2.2. Coverage data

3.2.3. Run-to-section

3.2.4. Globals

3.3. LSP features overview table

4. Implementation

4.1. The lsp4j library

4.2. Architecture

4.2.1. Truffle-API + Graal-SDK

4.2.2. Launcher, Instrument, Server

4.2.3. Thread with entered Context

4.3. What a TruffleLanguage needs to implement

The onInputValue method can be used in combination with source section filters to intercept very fine-grained execution events such as intermediate values used by language expressions. The intermediate values that are accessible to the instrumentation framework greatly depend on the instrumentation support provided by each language.

4.3.1. Expression- and Statement-Tag

4.3.2. findSourceLocation()

4.3.3. findTopScopes()

4.3.4. findLocalScopes()

4.3.5. findMetaObject()

4.3.6. getCompletionTriggerCharacters()

4.3.7. boxPrimitive()

4.3.8. parseInline()

4.3.9. GetSignature custom message

4.4. LSP features implementation - TextDocumentService

4.5. LSP features implementation - WorkspaceService

4.6. LSP client config

4.6.1. VSCode extension exmaple + Limitations

5. Experience report

5.1. Between IDE and Language Workbench

Language Workbench focus/aim: Efficient language (DSL) development, performance is not main focus vs. Graal/Truffle aim: Efficient execution

Assumption: Language Workbenches use static information only, our Truffle-based solution uses dynamic type information

5.2. Completion response time

5.3. Limitations

own chapter? or in cha:implementation?

5.3.1. VSCode: Need to define supported languages in extension

5.3.2. VSCode: Syntax highlighting defined at client-side

5.3.3. Server: Only TCP (connecting to already running server), no std-in/-out impl

5.3.4. Truffle: Only first syntax error is reported. Lang parsers fail on first error

6. Related Work

6.1. Graal

[8]

6.2. Squimera

[6] [5]

6.3. Language Workbenches

[1]

6.4. Run-time type collection

[3]

7. Conclusion and Future Work

Bibliography

[1] Sebastian Erdweg, Tijs Van Der Storm, Markus Völter, Meinte Boersma, Remi Bosman, William R Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, et al. "The State of the Art in Language Workbenches". In: *International Conference on Software Language Engineering*. Springer. Springer International Publishing, 2013, pp. 197–217. DOI: 10.1007/978-3-319-02654-1_11.

[2] Matthias Grimmer, Chris Seaton, Roland Schatz, Thomas Würthinger, and Hanspeter Mössenböck. "High-performance cross-language interoperability in a multi-language runtime". In: *ACM SIGPLAN Notices*. Vol. 51. 2. ACM. 2015, pp. 78–90. DOI: 10.1145/2936313.2816714.

[3] Michael Haupt, Michael Perscheid, and Robert Hirschfeld. "Type harvesting: a practical approach to obtaining typing information in dynamic programming languages". In: *Proceedings of the 2011 ACM Symposium on Applied Computing*. ACM. 2011, pp. 1282–1289. DOI: 10.1145/1982185.1982464.

[4] Microsoft. *Language Server Protocol*. https://microsoft.github.io/language-server-protocol/. Visited on 2018-09-24. 2017.

[5] Fabio Niephaus, Tim Felgentreff, Tobias Pape, and Robert Hirschfeld. "Squeak Makes a Good Python Debugger: Bringing Other Programming Languages Into Smalltalk's Tools". In: *Companion to the first International Conference on the Art, Science and Engineering of Programming*. ACM. 2017, p. 27. DOI: 10.1145/3079368.3079402.

[6] Fabio Niephaus, Tim Felgentreff, Tobias Pape, Robert Hirschfeld, and Marcel Taeumel. "Live Multi-language Development and Runtime Environments". In: *arXiv preprint arXiv:1803.10200* 2.3 (Mar. 2018). DOI: 10.22152/programming-journal.org/2018/2/8.

[7] Oracle. *GraalVM*. https://www.graalvm.org/. Visited on 2018-10-08. 2018.

[8] Chris Seaton, Michael L Van De Vanter, and Michael Haupt. "Debugging at Full Speed". In: *Proceedings of the Workshop on Dynamic Languages and Applications*. ACM. ACM Press, 2014, pp. 1–13. DOI: 10.1145/2617548.2617550.

[9] Michael L Van De Vanter. "Building debuggers and other tools: we can have it all". In: *Proceedings of the 10th Workshop on Implementation, Compilation, Optimization of Object-Oriented Languages, Programs and Systems*. ACM. 2015, p. 2. DOI: 10.1145/2843915.2843917.

Appendix A.

First Unimportant stuff.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbständig verfasst sowie keine anderen Quellen und Hilfsmittel als die angegebenen benutzt habe.

Berlin, den 10. Oktober 2018

Daniel Stolpe

Todo list

Is this already implementation? 5
own chapter? or in cha:implementation? 9